Apply filters to SQL queries

Project description

My company is currently working on strengthening system security. In my role, I help keep our system safe, look into potential security issues, and make sure employee computers receive necessary updates. Below are a few real-world examples showing how I used SQL queries with filters to tackle these tasks.

Retrieve after hours failed login attempts

We recently flagged a potential security incident that happened after normal business hours (after 18:00). To figure out what happened, I needed to check every failed login attempt made after hours.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_time > '18:00' AND success = FALSE;
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142  |       0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50  |       0 |
```

How I filtered the Data:

- I selected all records from the log_in_attempts table.
- I added a WHERE clause using the AND operator to combine two conditions:
    1. login_time > '18:00' filters for logins after 6:00 PM.
    2. success = FALSE narrows the results strictly to failed attempts.

Running this query gave us a clear list of all suspicious, after-hours login failures to investigate.

Retrieve login attempts on specific dates

After noticing suspicious activity on May 9, 2022, I needed to review all login attempts from both May 9th and the day before (May 8th).

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       0 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
```

How I filtered the Data:

- I pulled all data from the log_in_attempts table.
- I used a WHERE clause with an OR operator so the query would return matches for either date:
    1. login_date = '2022-05-09' grabs logins from May 9th.
    2. login_date = '2022-05-08' grabs logins from May 8th.

This gave us a complete picture of user activity across that two-day window.

Retrieve login attempts outside of Mexico

While reviewing login records, I noticed an unusual pattern involving logins outside of Mexico that calls for further investigation.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+----------------+---------+
| event_id | username | login_date | login_time | country | ip_address     | success |
+----------+----------+------------+------------+---------+----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       0 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       0 |
```

How I filtered the Data:

- I queried the log_in_attempts table using a WHERE clause with NOT and LIKE.
- In our database, Mexico is logged as either MEX or MEXICO.
- By using NOT country LIKE 'MEX%', the % wildcard matches any country name starting with "MEX" , and NOT excludes them.

This cleanly filtered out all Mexican logins and returned only international attempts.

Retrieve employees in Marketing

Our team needed to roll out computer updates for Marketing employees working in our East building, so I had to pull a list of their specific machines.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%';
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
+-------------+--------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
```

How I filtered the Data:

- I selected all records from the employees table.

- I applied a WHERE clause using AND to meet both criteria:
    1. department = 'Marketing' specifies the team.
    2. office LIKE 'East%' captures any office location starting with "East" (since office values include room numbers).

This produced an exact list of the computers needing updates.

Retrieve employees in Finance or Sales

Next, we had a separate security update targeted specifically at employees in Finance and Sales. I needed to gather machine details for everyone in those two departments.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Finance' OR department = 'Sales';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
```

How I filtered the Data:

- I queried the employees table.
- I used a WHERE clause with OR: department = 'Finance' OR department = 'Sales'.
- Using OR instead of AND ensured that employees belonging to either department were included, giving us the full target list.

Retrieve all employees not in IT

Finally, we needed to run a general security update across the entire company, excluding members of the IT department (who manage their updates separately).

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE NOT department = 'Information Technology';
+-------------+--------------+----------+-----------------+-------------+
| employee_id | device_id    | username | department      | office      |
+-------------+--------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
```

How I filtered the Data:

- I queried the employees table.
- I used WHERE NOT department = 'Information Technology' to filter out the IT team while returning everyone else in the company.

Summary

By using SQL filters, I was able to efficiently locate the exact login and employee records required for our security tasks. Working across the log_in_attempts and employees tables , I leveraged logical operators (AND, OR, NOT) alongside pattern-matching tools like LIKE and the % wildcard. This allowed me to pinpoint security incidents and streamline our hardware update process.